

SkillSwap NSU: what the live link runs, and where the database actually lives

Repository github.com/mahedi-123/SkillSwapNSU • Live site mahedi-123.github.io/SkillSwapNSU/

A student skill-exchange platform for North South University. No money is involved anywhere in the system.

This note exists because the submitted link behaves in a way that looks impossible at first glance. GitHub Pages is a static host — it serves files and runs no code of its own — yet the site accepts a sign-in, creates an account, accepts an exchange request, books a session and publishes a review. The purpose of the following pages is to state plainly what is really happening in the browser, what is *not* happening on any server, and where the graded database work sits in relation to it.

1 What GitHub Pages can and cannot run

GitHub Pages is a file server. When a browser asks for a URL, Pages locates a matching file in the repository and returns its bytes unchanged. It does not start a process, it does not have an interpreter attached to the request, and it does not have a database engine behind it. There is exactly one thing on that host that could be called “running”, and it is the Jekyll site generator — which this project deliberately switches off with the empty `.nojekyll` file at the repository root, so the folders are published verbatim.

Everything the visitor experiences as behaviour is therefore executed by the visitor's own browser, after the files arrive. HTML, CSS and JavaScript are client-side technologies: the browser is their runtime. Python, Flask, MySQL and PHP are server-side technologies: they need a process running on a machine that the request reaches. GitHub Pages offers no such process, which is why the following division is absolute rather than a matter of configuration.

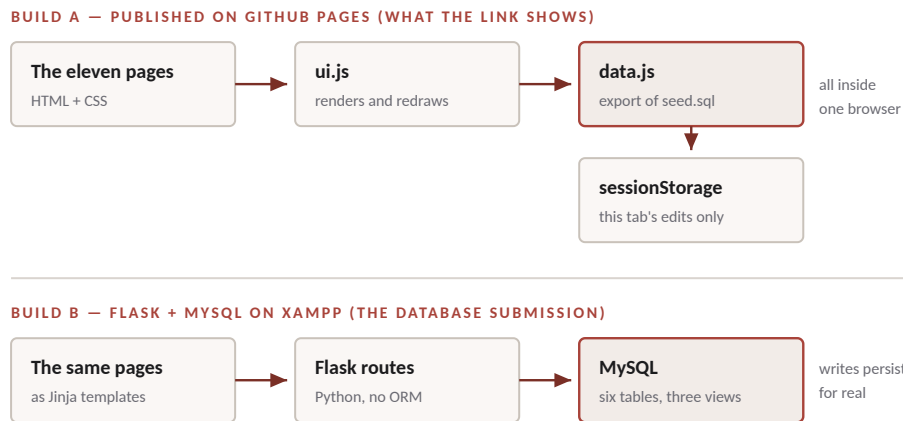
LAYER	WHERE IT EXECUTES	AVAILABLE ON GITHUB PAGES
HTML, CSS	The visitor's browser	Yes — this is exactly what Pages is for
JavaScript	The visitor's browser	Yes, with no server to call back to
Python / Flask	A server process	No — Pages never executes code
MySQL / MariaDB	A database server	No — nothing is listening on port 3306
PHP, phpMyAdmin	A server process	No

So the honest answer to “can GitHub run the database part?” is no, and this project never claims otherwise. What it does instead is described next.

2 The project ships in two builds from one set of pages

The eleven screens — landing page, sign in, register, dashboard, search, profile, edit profile, requests, sessions, reviews and the admin console — exist once, as ordinary HTML. What

changes between the two builds is only where the rows come from and where a write goes.



The markup is shared. Only the right-hand side of each row differs.

3 What the live link is actually doing

The published build reads its rows from `static/js/data.js`, a plain JavaScript file holding the same records that `database/seed.sql` inserts into MySQL. The browser loads that file, and `static/js/ui.js` joins and filters those arrays in memory to draw each screen. A search by skill and department, the match list on the dashboard, the rating average under a student's name — each is the JavaScript equivalent of the corresponding SQL query, computed on the client.

Writes are handled by a small operation log rather than by a server. When a button is pressed, the page records a compact description of the change — the operation type, the row identifier, the new value — appends it to an array, applies it to the in-memory rows, and asks the screen to redraw. That array is stored in the browser's `sessionStorage` under the key `skillswap.demo.ops`. Every page load replays the log from the start, which is why a request accepted on the requests screen is still accepted when the dashboard is opened a moment later. It looks like persistence, and within one browser tab it behaves like persistence, but nothing has left the machine: no request was sent, no row was written, and another visitor opening the same URL sees the untouched seed data.

Two things clear the log and restore the seeded state exactly: the *Reset demo data* control in the top bar, and closing the tab, because `sessionStorage` is scoped to the tab and discarded with it.

THE DETAIL WORTH SHOWING IN THE VIVA

Every write action raises a toast that names the SQL statement it stands for. Accepting request 12 reports `UPDATE exchangerequests SET status = 'Accepted' WHERE request_id = 12`; publishing a review reports the matching `INSERT INTO reviews (...)`. The toast is not decoration — it is the statement the Flask build executes at that exact point, so the interface documents its own data layer while it is being demonstrated.

Sign in and registration, specifically

These are the two actions that most look like they require a server, so they are worth stating precisely.

Signing in looks the submitted email up in the `USERS` array and compares the password against the literal `password123`, which every seeded account shares. On success the matched `user_id` is written to `sessionStorage`, and the rest of the site reads that value to decide whose dashboard, requests and sessions to show. In the Flask build the identical value lives in `session['user_id']` on the server, set after Werkzeug has checked the submitted password against the stored hash.

Registration appends a genuine new row to the in-memory `users` table plus two rows to `userskills`, which is why a newly created account appears immediately in search and in the admin list. The chosen password is deliberately discarded rather than stored: a browser cannot produce a Werkzeug hash, and keeping a plain-text password in client memory would be a worse lesson than not keeping it at all. The new account therefore also signs in with `password123`, and the register page says so on screen.

4 Why the screens still match the database exactly

`data.js` is not sample data invented for the demo. It is a direct export of `seed.sql`, and the row counts are identical on both sides, so any figure read off the live site can be checked against the same figure in phpMyAdmin.

TABLE	ROWS IN SEED.SQL	ROWS IN DATA.JS
<code>users</code>	50	50
<code>skills</code>	50	50
<code>userskills</code>	261	261
<code>exchangerequests</code>	49	49
<code>sessions</code>	34	34
<code>reviews</code>	36	36

The admin console is the quickest place to see this: its header reads 50 students and 50 skills, and the student table lists all fifty rows.

5 Where the graded database work actually lives

The database submission is not the website. It is the four files under `database/` in the same repository, which are imported into MySQL through XAMPP and examined there. The single file `database/skillexchange_full.sql` is the easiest route: opening phpMyAdmin, choosing the Import tab and running that one file drops any earlier copy, recreates all six tables with their keys, constraints and indexes, creates the three views, and inserts the sample data. Importing `schema.sql` and then `seed.sql` separately produces the same result.

FILE	CONTENTS
database/schema.sql	Six tables, primary and foreign keys, CHECK and UNIQUE constraints, indexes, three views
database/seed.sql	Sample data for every table
database/queries.sql	Every SQL feature the lab requires, numbered A1 to J3
database/skillexchange_full.sql	Schema and seed combined, for a one-step import

The design is six tables in third normal form: `users` and `skills` resolved through the `userskills` bridge table, `exchangerequests` holding one proposed trade with a sender and a receiver, `sessions` belonging to a request, and `reviews` belonging to a session. Referential behaviour is explicit — rows that are meaningless without their parent cascade on delete, while a skill cannot be deleted while an exchange request still points at it. Averages are never stored on `users`; they are derived through the `v_user_ratings` view, which is what keeps the design in 3NF.

`queries.sql` is grouped so each requirement can be found without reading the whole file. Joins across up to five tables sit in group B, outer joins in C, aggregation and `HAVING` in D, the six subquery forms in E, the parameterised searches the interface actually uses in F, views in G, indexes with `EXPLAIN` proof in H, the write statements in I, and the reporting queries in J. Every write query in group I is wrapped in a transaction that rolls back, so running the file against the seeded database leaves the sample data undisturbed.

6 How the same interface becomes the Flask build

The static build was written so that the move to Flask changes the data layer and not the markup. Each JavaScript action has one server-side counterpart, and the HTML around it stays as it is — the arrays become Jinja variables, and the client-side `commit()` call becomes a form POST that runs the parameterised statement already named in the toast.

ACTION IN THE INTERFACE	STATIC BUILD (GITHUB PAGES)	FLASK BUILD (XAMPP)
Draw a list or a profile	Filter the arrays in <code>data.js</code>	<code>SELECT</code> with joins, rows passed to the template
Sign in	Match the email in <code>USERS</code> , store the id in <code>sessionStorage</code>	Werkzeug checks the hash, id stored in <code>session['user_id']</code>
Register	Append a row in memory, password discarded	<code>INSERT INTO users</code> with <code>generate_password_hash</code>
Send a request	Record a <code>newRequest</code> operation	<code>INSERT INTO exchangerequests</code>
Accept or reject	Record a <code>reqStatus</code> operation	<code>UPDATE exchangerequests SET status = %s</code>
Book or complete a session	Record a <code>newSession</code> or <code>sessStatus</code> operation	<code>INSERT INTO sessions</code> / <code>UPDATE sessions</code>
Leave a review	Record a <code>newReview</code> operation	<code>INSERT INTO reviews</code>
Admin deletes a student	Remove the dependent rows in the same order the schema cascades	<code>DELETE FROM users</code> , and <code>ON DELETE CASCADE</code> does the rest

The admin deletion is the clearest illustration. In the browser the code walks reviews, then sessions, then requests, then `userskills`, then the student — deliberately mirroring the cascade the schema declares — so that the interface shows the same consequence the database would enforce. In the Flask build that loop disappears entirely and a single `DELETE` statement produces the identical result, because the constraint is doing the work.

The planned stack for that build is Flask with Blueprints on Python 3, MySQL on XAMPP through `mysql-connector-python`, and Flask sessions with Werkzeug hashing for authentication. No ORM is used anywhere: every query is written by hand and parameterised, which is both the point of a Database Systems Lab project and the defence against SQL injection.

7 What the live link does and does not prove

Stated plainly, so nothing is read into it that is not there. The link demonstrates that the eleven screens are complete and navigable, that the interface is built against the real seeded records rather than placeholder text, that every interaction the finished product needs is wired up and produces the correct consequence on screen, and that each of those interactions is annotated with the SQL statement behind it.

The link does not demonstrate a running database. Nothing is persisted beyond the tab, two visitors never see each other's changes, passwords are not hashed because no hash can be computed or stored without a server, and no constraint is being enforced by MySQL — the cascade the admin console shows is JavaScript imitating the schema, not the schema itself. Those properties are demonstrated in the database submission, by importing the SQL into XAMPP and running `queries.sql` against it.

8 Verifying all of this in about five minutes

Opening `mahedi-123.github.io/SkillSwapNSU/` and signing in as `mahedi.shakib@northsouth.edu` with `password123` reaches the dashboard; the *Fill demo credentials* button on the sign-in page enters a working pair. Accepting a pending request on the requests screen changes its status and raises a toast naming the `UPDATE` statement, and opening the dashboard afterwards shows the change has carried across pages. Pressing *Reset demo data* in the top bar, or simply closing the tab, restores the seeded state — which is the point being made: the change never left the browser.

For the database side, starting Apache and MySQL in the XAMPP control panel, opening `localhost/phpmyadmin` and importing `database/skillexchange_full.sql` produces the six tables with the row counts listed in section 4. Pasting `database/queries.sql` into the SQL tab runs every required feature in turn, and because the write queries are wrapped in transactions that roll back, the sample data is unchanged afterwards. Comparing any figure from those results against the same figure on the live admin console shows the two sides agree.

Finally, `preview.html` in the repository is a single generated file containing all eleven screens with a tab strip, produced from the real pages by `tools/build-preview.js`. It opens by double-clicking, with no server and no internet, and is the fastest way to review the whole interface offline.

